

Algorytmy i złożoność obliczeniowa - Projekt

Sprawozdanie

Zadanie projektowe nr 1

Analiza algorytmów sortowania i wpływu struktury danych na czas wykonania

Mykhailo Tsonyev nr albumu 284497

Prowadzący kurs: Termin zajęć: TN wtorek 9:15 Termin oddania zadania: 12 maja 2026

1. Wstęp

Celem projektu było samodzielne zaimplementowanie i przebadanie trzech algorytmów sortowania w zakresie wymagany na ocenę 3.0. Program miał poprawnie sortować dane rosnąco, weryfikować wynik, mierzyć czas samego sortowania oraz zapisywać wyniki badań do pliku CSV.

Zakres funkcjonalny obejmował trzy algorytmy: sortowanie koktajlowe, przez scalanie oraz przez wstawianie. Zaimplementowano również dwie liniowe struktury danych: tablicę dynamiczną i listę jednokierunkową. Badania wykonano dla przypadków A, B i C wymaganych dla oceny 3.0.

Program uruchamiano na komputerze MacBook Air M1 z 8 GB pamięci RAM. Dane eksperymentalne pochodzą z pliku `results/research_data.csv`, który zawiera 35 badań, a każde badanie składa się z 50 iteracji.

2. Opis implementacji

Podstawą projektu są dwie własne struktury danych. `DynamicArray<T>` przechowuje elementy w ciągłym buforze i zapewnia szybki dostęp indeksowy, natomiast `SinglyLinkedList<T>` przechowuje elementy w węzłach połączonych wskaźnikami. Obie struktury są alokowane dynamicznie i nie korzystają z kontenerów STL takich jak `std::vector` czy `std::list`.

Implementacja używa szablonów, dzięki czemu te same algorytmy mogą pracować na wielu typach danych. W części badawczej wykorzystano typ podstawowy `int`, a w badaniu C dodatkowo `float` oraz `unsigned int`. Szablony są szczególnie użyteczne przy zachowaniu wspólnego interfejsu dla tablicy i listy.

W programie zaimplementowano trzy algorytmy:

- `CocktailSort`, który wykonuje naprzemienne przejścia od lewej i od prawej strony.
- `MergeSort`, który dzieli dane na części, a następnie scala je w kolejności rosnącej.
- `InsertionSort`, który buduje uporządkowany fragment danych przez kolejne wstawianie elementów.

Dla listy jednokierunkowej część algorytmów ma osobne wersje dostosowane do braku taniego dostępu indeksowego. `MergeSort` dla listy przepina wskaźniki pomiędzy węzłami, a `InsertionSort` tworzy uporządkowaną listę przez kolejne wstawianie w odpowiednie miejsce. Dzięki temu implementacja lepiej odpowiada właściwościom badanej struktury.

Program zawiera też walidację parametrów wejściowych, sprawdzenie poprawności wyniku przez funkcję `isSortedAscending(...)` oraz pomiar czasu przy pomocy `std::chrono`. Mierzony jest wyłącznie czas sortowania, bez generowania danych i bez zapisu wyników do pliku.

Przykładowy fragment odpowiedzialny za pomiar czasu sortowania:

```
auto start = std::chrono::steady_clock::now();
if (runBenchmarkSort<T, Structure>(values) != 0)
{
    std::cerr << "ERROR: sorting failed during benchmark iteration " << iteration
    << ".\n";
    return 1;
}
auto end = std::chrono::steady_clock::now();

long long duration = std::chrono::duration_cast<std::chrono::microseconds>(end -
start).count();
```

3. Metodyka badań

W niniejszym sprawozdaniu przez *badanie* rozumiany jest jeden kompletny benchmark z ustalonym zestawem parametrów: algorytmem, strukturą danych, typem danych, rozkładem wejścia i rozmiarem danych. Przez *iterację* rozumiany jest pojedynczy powtórzony pomiar w ramach tego samego badania.

Każde badanie zostało wykonane 50 razy. W pliku CSV zapisano surowy czas każdej iteracji, a w ostatnim wierszu danego bloku dopisano wartości `min_us`, `avg_us` i `max_us`. Taka organizacja danych jest wystarczająca, ale wymaga ostrożnej analizy, ponieważ niektóre badania mają identyczne parametry logiczne. Z tego powodu wyniki nie były grupowane jedynie po kolumnach `algorithm`, `structure`, `data_type`, `distribution` i `size`, lecz według rzeczywistej kolejności 35 bloków po 50 wierszy.

Zakres eksperymentów:

- Badanie A: 24 zadania, czyli każda kombinacja 3 algorytmów, 2 struktur i 4 rozmiarów danych.
- Badanie B: 8 zadań dla MergeSort, obu struktur i czterech rozkładów wejścia.
- Badanie C: 3 zadania dla MergeSort, struktury array, rozmiaru 25000 i typów int, float, unsigned int.

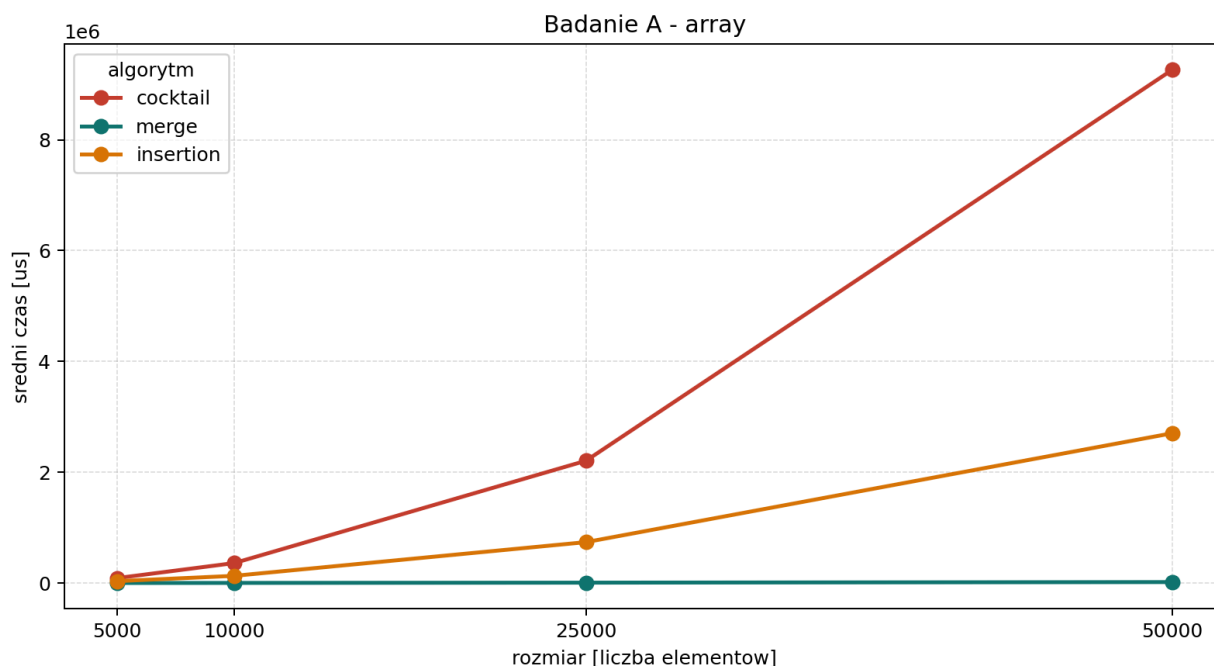
Do przygotowania wyników użyto skryptu Python, który:

- waliduje, że plik wejściowy zawiera dokładnie 1750 rekordów danych,
- dzieli rekordy na 35 bloków po 50 iteracji,
- zapisuje tabele zbiorcze dla badań A, B i C,
- generuje wykresy PNG do wykorzystania w sprawozdaniu.

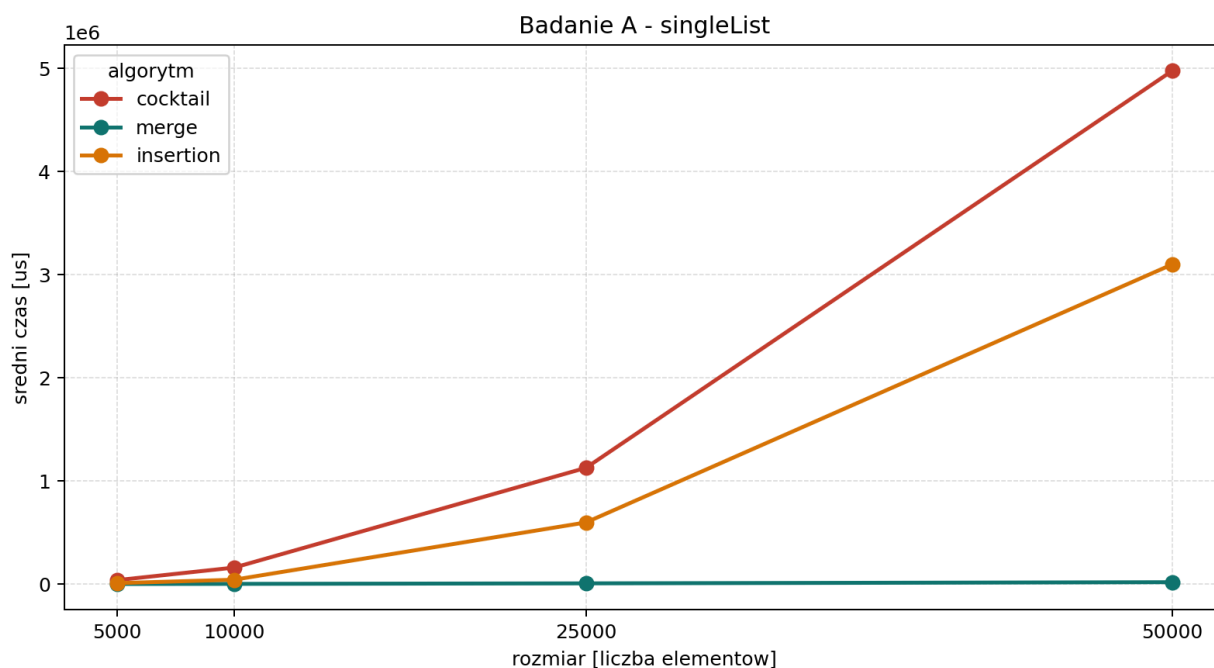
4. Wyniki badań

4.1. Badanie A

Badanie A sprawdza wpływ liczebności zbioru na czas działania algorytmów. Na wykresach widać wyraźnie, że MergeSort skaluje się znacznie lepiej niż CocktailSort i InsertionSort, zwłaszcza dla większych rozmiarów danych.



Rysunek 1: Badanie A dla struktury `array`.

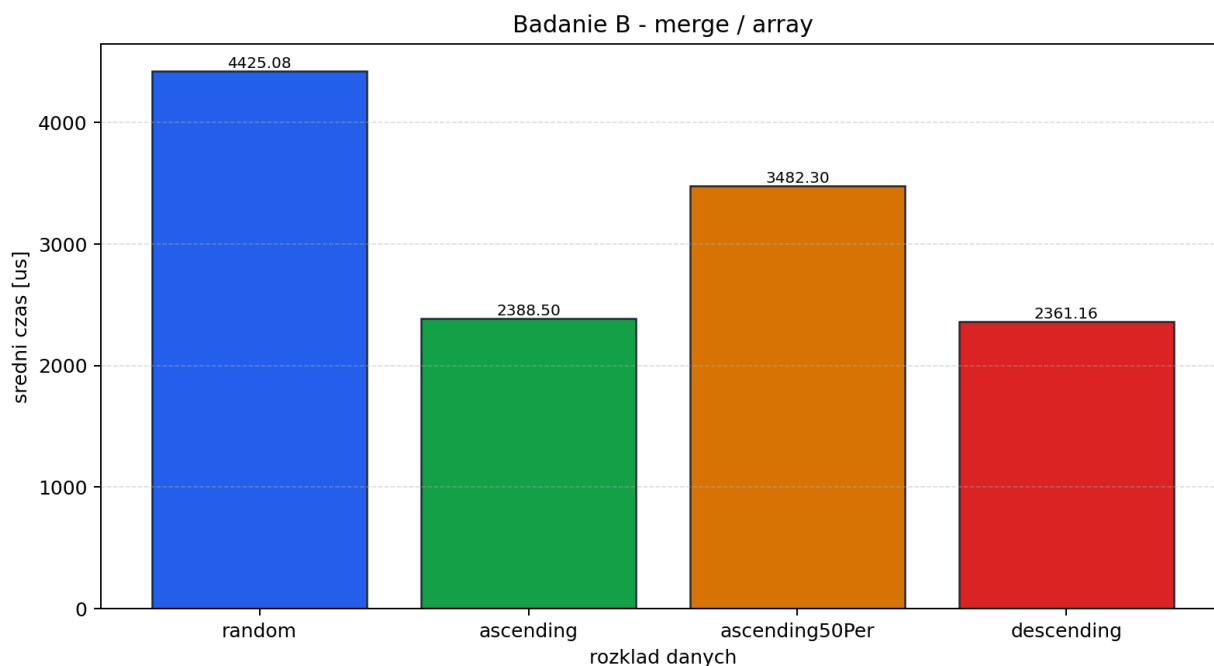


Rysunek 2: Badanie A dla struktury `singleList`.

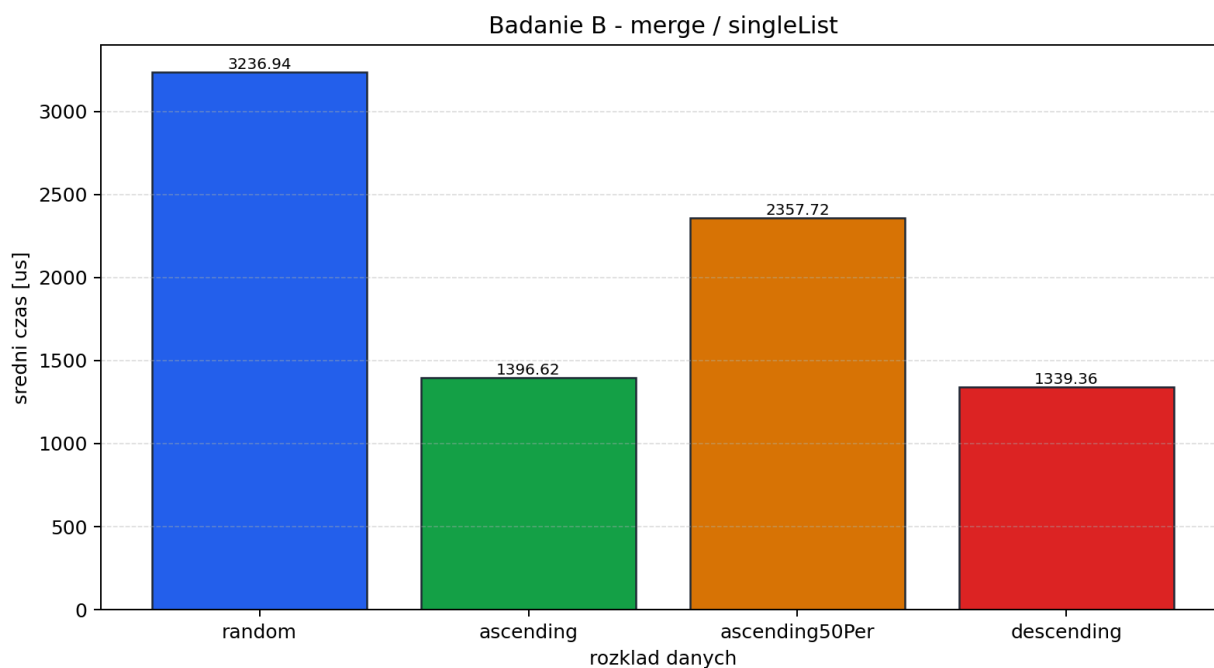
Przykładowo dla `array` i rozmiaru `50000` średni czas MergeSort wyniósł `17923.3` us, podczas gdy CocktailSort osiągnął `9263002.6` us, a InsertionSort `2703807.76` us. Oznacza to różnicę rzędu kilku rzędów wielkości pomiędzy algorytmem $O(n \log n)$ a algorytmami kwadratowymi.

4.2. Badanie B

Badanie B dotyczy wpływu rozkładu danych wejściowych. Dla MergeSort zauważalna jest niewielka zależność czasu od tego, czy dane są losowe, rosnące, częściowo uporządkowane czy malejące. Największa różnica pojawia się między danymi losowymi a już uporządkowanymi, ale skala zmian jest nadal umiarkowana.



Rysunek 3: Badanie B dla MergeSort i struktury array.

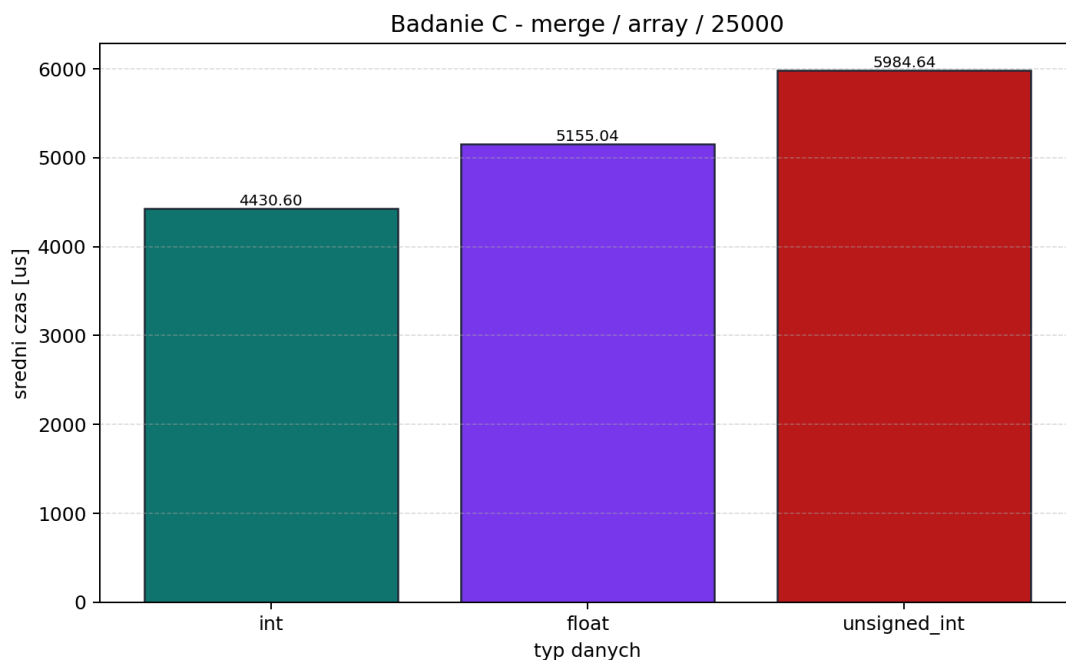


Rysunek 4: Badanie B dla MergeSort i struktury singleList.

Dla array średni czas dla rozkładu losowego wyniósł 4425.08 us, natomiast dla danych rosnących 2388.5 us, dla 50% uporządkowania 3482.3 us, a dla danych malejących 2361.16 us. Analogiczny trend widać dla listy jednokierunkowej.

4.3. Badanie C

Badanie C analizuje wpływ typu danych przy zachowaniu tego samego algorytmu, tej samej struktury oraz tego samego rozmiaru wejścia. Wyniki pokazują, że dominujący wpływ ma sam algorytm i organizacja danych, a nie drobne różnice pomiędzy `int`, `float` i `unsigned int`.



Rysunek 5: Badanie C dla MergeSort, array i rozmiaru 25000.

Średnie czasy wyniosły odpowiednio:

- int: 4430.6 us
- float: 5155.04 us
- unsigned int: 5984.64 us

5. Analiza wyników

Uzyskane rezultaty są zgodne z oczekiwaniami wynikającymi z teorii oraz z rzeczywistej implementacji programu. CocktailSort i InsertionSort wykazują zachowanie kwadratowe, dlatego wraz ze wzrostem rozmiaru danych ich czas gwałtownie rośnie. Jest to szczególnie widoczne dla rozmiaru 50000, gdzie oba algorytmy są wielokrotnie wolniejsze od MergeSort.

W przypadku MergeSort zarówno dla tablicy, jak i dla listy jednokierunkowej zachowana jest złożoność asymptotyczna $O(n \log n)$. Dla tablicy algorytm korzysta z jednego bufora pomocniczego, a dla listy sortuje dane przez dzielenie listy i scalanie poprzez przepinanie wskaźników. To sprawia, że MergeSort dobrze odpowiada właściwościom obu badanych struktur.

CocktailSort dla listy nie korzysta z klasycznego `nodeAt(index)` przy każdej operacji, lecz najpierw buduje tablicę wskaźników do węzłów. Zmniejsza to koszt dostępu do elementów, ale nie zmienia ogólnej natury algorytmu: liczba porównań i zamian nadal rośnie kwadratowo. Z kolei InsertionSort dla listy działa przez wstawianie w posortowaną część listy, dlatego również zachowuje dominująco kwadratowy charakter. Dla tablicy najlepszy przypadek może zbliżać się do $O(n)$, ale w sprawozdaniu należy interpretować ten algorytm głównie przez jego średnie i najgorsze zachowanie.

Rozkład danych z badania B nie zmienia jakościowo wyniku MergeSort, bo sam algorytm jest odporny na wstępne uporządkowanie danych znacznie bardziej niż algorytmy kwadratowe. Różnice czasów między rozkładami można tłumaczyć m.in. szczegółami porównań, lokalnością pamięci oraz kosztem pracy na konkretnych strukturach.

W badaniu C wpływ typu danych okazał się niewielki. Jest to spójne z założeniem, że dla implementacji MergeSort najważniejsze są liczba operacji porównania i przenoszenia elementów oraz sposób przechowywania danych, a nie sam wybór pomiędzy `int`, `float` i `unsigned int`.

6. Wnioski

Projekt spełnia wymagania na ocenę 3.0: zaimplementowano trzy wymagane algorytmy, dwie wymagane struktury danych oraz przeprowadzono komplet badań A, B i C. Program poprawnie waliduje parametry, sprawdza wynik sortowania i zapisuje powtarzalne wyniki pomiarów do pliku CSV.

Najważniejszym wnioskiem z badań jest wyraźna przewaga MergeSort dla większych instancji danych. Algorytmy kwadratowe mogą być poprawne i proste implementacyjnie, ale ich skalowanie jest znacznie słabsze. Wyniki są zgodne z teorią, a jednocześnie dobrze pokazują, że szczegóły implementacyjne struktur danych wpływają na realny czas wykonania.

7. Literatura

- [1] Maciej M. Sysło, *Algorytmy*, Helion, 2016.
- [2] L. Banachowski, K. Diks, W. Rytter, *Algorytmy i struktury danych*, Wydawnictwo Naukowe PWN, Warszawa, 2018.
- [3] R. Lazarus, R. Frank, *A High-Speed Sorting Procedure*, Communications of the ACM, 3(1), 1960.
- [4] Marcin Kasprowicz, *Złożoność obliczeniowa algorytmu*, algorytm.edu.pl, dostęp online.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Wprowadzenie do algorytmów*, PWN, 2022.